Coherence–Rupture–Regeneration as a Left Kan Extension

Causal gating, convex regeneration and the carrier threshold

Alexander Sabine · Active Inference Institute · alexander@activeinference.institute

October 2026

Abstract. Coherence–Rupture–Regeneration (CRR) describes persisting systems that accumulate Fisher–Rao arc on a statistical carrier, are cut at the first passage of a half-turn of net travel, and regenerate from an age-weighted mean of completed occasions. I give this structure a categorical and geometric formulation and establish five results.

1. **Causal gating is a left Kan extension.** The rule that only completed occasions contribute to the present is the left Kan extension of a diagram of completed occasions along their inclusion into time. Two consequences follow:
    - along a single history, the present holds exactly the latest completed occasion;
    - across interacting systems, it is the colimit, or gluing, of every past in the causal cone.
2. **Regeneration is an algebra map for the distribution monad.** For convex content spaces, the convex-algebra law guarantees that recursive regeneration equals regeneration from the whole record.
3. **The guarantee fails on curved content.** On the circle, recursion and the whole-record Fréchet mean agree whenever the record lies within a half-circle, but can differ by more than a radian beyond it. Regeneration on curved content therefore depends on the order of history, not only on its summary.
4. **The threshold depends on the carrier.**
    - The Fisher–Rao diameter of a probability simplex is exactly π, attained between vertices. Bonnet–Myers bounds it only by 2π.
    - The current cut threshold $\Omega = 1/a$ is set by the half-turn a of the chosen carrier: for example $\Omega \approx 0.716$ for a three-outcome circular family.
5. **Weights and timing are fixed separately.** Geometric recency weights are the maximum-entropy weights for a given mean age. First-passage variability is set by dynamics, as $\mathrm{CV} = \sqrt{(\sigma^2/(\mu\Omega))}$, and is not fixed by geometry.

All results were preregistered and confirmed on fresh random instances.

Keywords: Kan extension; colimit; causal order; distribution monad; convex algebra; Fréchet mean; Fisher–Rao geometry; maximum entropy; first passage.

1. Introduction

Many systems integrate information until they must commit: a perceptual decision, a cell’s fate, a learner’s revised belief. CRR formalises this as three operations:

- **coherence:** accumulated Fisher–Rao arc;
- **rupture:** a cut at a threshold;
- **regeneration:** reconstruction of the next occasion from completed ones.

Category theory offers a natural language for the integrative step. A left Kan extension assembles a functor's values by gluing everything that maps into a given object, which is the formal shape of "accumulating information from many sources".

An earlier formulation of this idea (Sabine, January 2026) proposed that CRR emerges from bounded left Kan extensions. It also derived a universal threshold $\Omega = \pi$ from the Bonnet–Myers theorem and a parameter-free timing variability. The present paper develops the categorical reading in the current CRR formulation:

- the cut is a first passage of net travel through a half-turn of the chosen carrier;
- regeneration is an age-weighted Fréchet mean;
- physical dynamics are added to the framework rather than derived from it.

This yields sharper statements about what the categorical structure does and does not determine.

2. Framework

- **Carrier and coherence.** A state is a point on a statistical carrier with the Fisher–Rao metric. Coherence C is accumulated arc, the chord C* is geodesic distance, and surplus is $S = C - C^*$.
- **The cut.** On an intrinsic circular carrier of circumference ℓ, a cut occurs at $t_{(n+1)} = \inf\{ t > t_n : s(t) - s(t_n) = a \}$, with $a = \ell/2$ and $\Omega = 1/a$.
- **Regeneration.** The seed of occasion n+1 is $m_{n+1} \in \operatorname{argmin}_y \Sigma_{j \le n} q^{n-j} d(y, \Phi_j)^2$, an age-weighted Fréchet mean of completed contents Φ_j, with $0 \le q < 1$. In flat coordinates it is the recursion $m_{n+1} = q\, m_n + (1 - q)\, \Phi_n$.
- **Completion.** An occasion contributes only once its cut has occurred.

3. Results

3.1 Causal gating as a left Kan extension

Let E be the set of completed occasions, partially ordered by causal precedence: within one system by time, and across systems by signals sent and received. Let $F : E \to \mathbf{C}$ assign to each occasion its content, in a cocomplete category **C** (for instance finite sets, with functions along the order). Let $i : E \to T$ include the occasions into an observer's time line.

Proposition 1. The left Kan extension of F along i is

$\mathrm{Lan}_i F(t) = \operatorname{colim} F(e)$, the colimit taken over the occasions $e \in E$ with $e \le t$.

Proof. This is the pointwise formula for left Kan extensions (Mac Lane 1971, ch. X). The comma category $(i \downarrow t)$ is the down-set of occasions causally preceding t. ∎

Three consequences make CRR's gating precise:

- **(a) Only the causal past matters.** $\mathrm{Lan}_i F(t)$ is unchanged by any alteration of F on occasions outside the causal past of t. This is the Heaviside gate.
- **(b) A single history.** If the down-set is a chain with a last element e*, the colimit is F(e*): the present holds exactly the latest completed occasion.
- **(c) Interacting systems.** If signals connect several systems, the down-set has no terminal element. The colimit then glues the pasts of all systems in the causal cone, identifying exactly what the signals identify.

Read as an account of the present, (c) says the present of an observer is the gluing of the completed pasts that reach it.

3.2 Regeneration as an algebra for the distribution monad

Convex sets are exactly the Eilenberg–Moore algebras of the finitely supported distribution monad D (Jacobs 2010). An algebra map $\alpha : D(Y) \to Y$ sends a weighted collection of points to their weighted mean.

Proposition 2. On a convex content space, regeneration is $m_{n+1} = \alpha(w_n)$, where w_n is the age-weighted distribution over completed contents. The flat recursion equals regeneration from the whole record.

Proof. The recursion applies α to a distribution over $\{m_n, \Phi_n\}$ in which m_n is itself α of the earlier record. The algebra law $\alpha \circ \mu = \alpha \circ D(\alpha)$ states that averaging averages equals averaging the flattened distribution, and the flattened weights are exactly $q^{n-j}(1 - q)$, together with q^{n+1} on the prior. ∎

Proposition 3. On the circle, which carries no global convex structure, the law can fail.

- If all contents lie within an open half-circle, arc-length coordinates are flat on their hull, and recursion and the whole-record Fréchet mean agree.
- Beyond a half-circle, the Fréchet mean can jump across the cut locus (Karcher 1977), and the recursive seed then depends on the order in which contents arrived.

So on curved content, regeneration is not a function of a fixed-size summary of the past. The order of history matters.

3.3 The threshold is fixed by the carrier

Proposition 4.

- **(a) The simplex.** The square-root map $p \mapsto 2\sqrt{p}$ embeds the probability simplex isometrically into a sphere of radius 2, with sectional curvature 1/4. The Fisher–Rao distance is $2 \arccos \Sigma\sqrt{(p_i q_i)}$, so the simplex has diameter exactly π, attained between vertices.

- **(b) Bonnet–Myers.** Applied to curvature 1/4, Bonnet–Myers gives only the bound 2π, which is not attained.
- **(c) The cut threshold.** In current CRR the threshold is not a universal constant but Ω = 1/a, with a half the circumference of the chosen intrinsic circular carrier. For the family $p_j(\theta) = [1 + 0.6 \cos(\theta - 2\pi j/3)]/3$, j = 0, 1, 2, the circumference is 2.794, so Ω ≈ 0.716.

3.4 Recency weights and timing

Proposition 5. Among weights on ages 0, …, n with a fixed mean age, entropy is maximised uniquely by geometric weights q^{age} (Jaynes 1957).

The same Lagrangian argument, applied to a fixed mean coherence instead of a fixed mean age, yields weights exponential in coherence. Which constraint is appropriate is a modelling choice; the current formulation uses age.

Proposition 6. If net travel accumulates as drift–diffusion with drift μ and noise σ, the time to a threshold Ω is inverse Gaussian with

$CV = \sqrt{(\sigma^2/(\mu\Omega))}$.

Timing variability is therefore set by the dynamics: any CV can be obtained by choosing σ.

4. Numerical confirmation

4.1 Design

Each proposition was implemented and checked on fresh random instances (study `KAN-CHECK-2026-10-28`).

Pipeline:

1. **Development.** Thresholds and tolerances were fixed on development draws.
2. **Freeze.** Code, scorer and predictions were frozen (FREEZE sha256 `5f30efc5…29b7`).
3. **Anchor.** The freeze was anchored by a pushed commit (`d88b628`) and an OpenTimestamps stamp.
4. **Holdout.** Only then was the confirmation draw made: random generator 3000 and first-passage seeds 3001–3006.
5. **Reproducibility.** Two reruns were byte-identical.

Instrument controls, scored before the freeze:

- the colimit routine reproduces a textbook pushout;
- a recursion with mismatched weights fails the flat law;
- averaging angles without wrapping fails on the circle.

4.2 Outcomes

All preregistered predictions held.

Check	Holdout outcome
Prop. 1(b): chain colimit equals the latest completed content	500 / 500 random diagrams
Prop. 1(a): colimit unchanged by edits outside the causal past	500 / 500 (247 of the pasts included other systems' occasions)
Prop. 2: flat recursion = whole-record mean	1,000 records; maximum error 0 to machine precision
Prop. 3: circle within a half-circle	588 records; maximum difference 2×10^{-8} rad (solver tolerance)
Prop. 3: circle beyond a half-circle	212 records; 36% differ by $> 10^{-3}$ rad; maximum 2.79 rad
Prop. 4(a,b): simplex diameter	largest sampled distance $3.097 \leq \pi$; vertex pairs exactly π; Bonnet–Myers bound 2π
Prop. 4(c): carrier threshold	circumference 2.794018; $\Omega = 0.7158$
Prop. 5: maximum-entropy ages	geometric weights beat 2,000 / 2,000 constrained perturbations
Prop. 6: timing	simulated CV within 2.6% of $\sqrt{(\sigma^2/(\mu\Omega))}$ across six settings (CV 0.21–0.65)

Figure 1 shows the threshold behaviour of Proposition 3. Within a half-circle the two regenerations coincide to solver precision. Beyond it, a substantial fraction diverge by up to π.

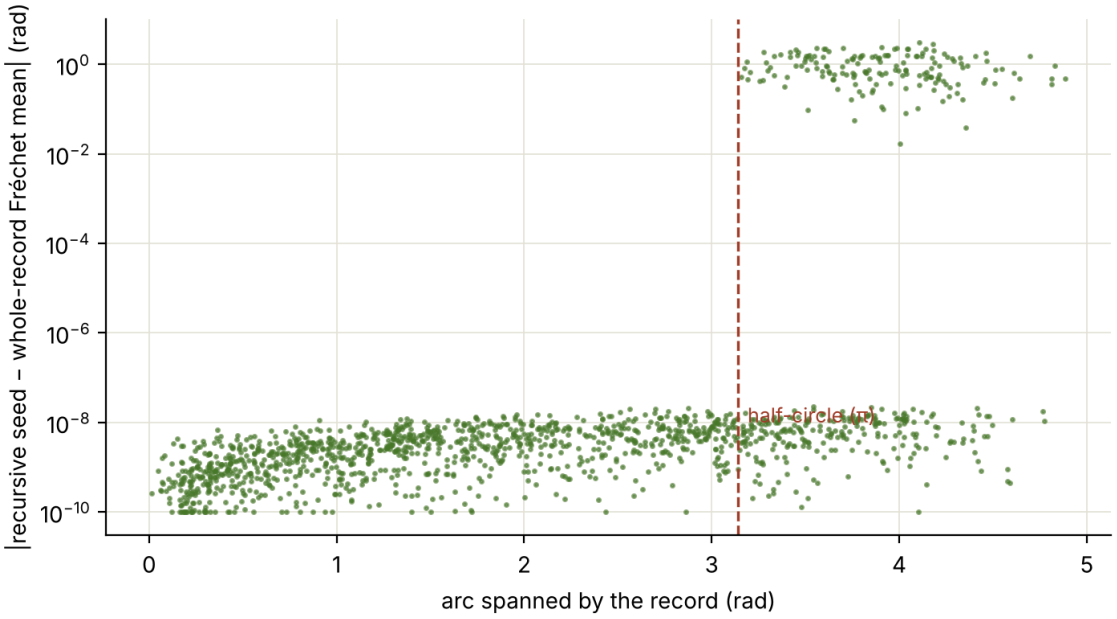


Figure 1. Difference between recursive regeneration and the whole-record Fréchet mean on the circle, against the arc spanned by the record (illustrative draws, not the holdout). Below a half-circle the two coincide; beyond it they can diverge by more than a radian.

5. Discussion

5.1 What the categorical reading contributes

The left Kan extension gives CRR's "only completed occasions contribute" an exact form, and extends it beyond a single system. The present of an observer is the colimit of the completed pasts in its causal cone:

- for one history, the latest completed occasion;
- for many, their gluing along the signals that connect them.

This states precisely, in categorical terms, the process view that what lies beyond a system's boundary is the settled past of other systems.

5.2 Where history exceeds its summary

Proposition 2 explains why regeneration can usually be computed recursively: on convex content, a running mean is a sufficient summary of the past.

Proposition 3 locates the exception. On curved content such as phases, orientations or cyclic states, the running seed and the whole-record mean part ways once the record spans more than a half-circle. Order then matters, and no fixed-size summary replaces the history.

This is the formal point at which CRR's emphasis on history goes beyond standard recursive filtering, and a natural target for empirical work on cyclic systems.

5.3 What geometry fixes and what it does not

Geometry fixes where a cut lies: half a turn of the chosen carrier, with diameter π for any simplex. It does not fix when cuts occur, nor how variable their timing is; those are set by the dynamics added to the framework.

5.4 Relation to the earlier formulation

The January 2026 formulation treated rupture as a "forced cocone" on a partially processed diagram. It derived $\Omega = \pi$ from Bonnet–Myers, and obtained $CV = 1/(2\pi)$ by setting $\sigma = 1/\sqrt{(2\pi)}$ in the expression of Proposition 6. It also cited a variability of 0.155 from human saltatory growth.

The present formulation differs as follows:

- **The threshold.** It replaces the Bonnet–Myers argument by the exact sphere embedding (Proposition 4). Bonnet–Myers bounds the simplex diameter but does not attain it.
- **Timing.** It treats timing variability as dynamical (Proposition 6).
- **The saltation source.** That study is Lampl, Veldhuis and Johnson (1992), on the saltatory pattern of infant growth.
- **The categorical content.** It is relocated from the cut to the gating of completed occasions (Proposition 1) and to regeneration (Propositions 2–3), where it carries exact consequences.

5.5 Limitations

- **The propositions assemble known mathematics** under CRR's definitions; the numerical checks confirm implementation, not empirical claims.
- **Proposition 3 was examined for the circle.** Other curved content spaces (spheres, rotation groups) are expected to behave similarly beyond their convexity radius, but were not tested.
- **The evidential status is preliminary.** The confirmation was preregistered and anchored, but run without an owner-signed tag or an independent operator. Its evidential ceiling under the Phoenix protocol is PASS-0.

6. Conclusion

In its current formulation, CRR has a clean categorical core.

- The gating of the present by completed occasions is a left Kan extension, and the present is the colimit of the pasts in its causal cone.
- Regeneration is an algebra map for the distribution monad, so it is recursive on convex content and order-dependent on curved content.
- The cut threshold is set by the carrier's half-turn, and timing is set by dynamics.

These results separate what CRR's structure determines from what must be supplied by the physics of a given system.

Data and code availability

All code is listed in full in the Appendix and is held in the Phoenix repository:

- the study: `Phoenix_Egg/studies/KAN-CHECK-2026-10-28/`;
- the receipts: `Phoenix_Egg/receipts/KAN-CHECK-2026-10-28/`.

Requirements: Python 3.13, numpy 2.5.3 and scipy 1.18.1, plus matplotlib for the figure. `sh run.sh RUNS_DIR` reproduces §4 in seconds, and `python3 -B figure.py` reproduces Figure 1.

References

1. S. Mac Lane, *Categories for the Working Mathematician*, Springer, 1971.
2. B. Jacobs, Convexity, duality and effects, in *Theoretical Computer Science (IFIP TCS 2010)*, IFIP AICT 323, Springer, 2010.
3. H. Karcher, Riemannian center of mass and mollifier smoothing, *Communications on Pure and Applied Mathematics* 30, 509–541, 1977.
4. S. Amari, *Information Geometry and Its Applications*, Springer, 2016.
5. S. B. Myers, Riemannian manifolds with positive mean curvature, *Duke Mathematical Journal* 8, 401–404, 1941.
6. E. T. Jaynes, Information theory and statistical mechanics, *Physical Review* 106, 620–630, 1957.

7. E. Schrödinger, Zur Theorie der Fall- und Steigversuche an Teilchen mit Brownscher Bewegung, *Physikalische Zeitschrift* 16, 289–295, 1915.
8. M. Lampl, J. D. Veldhuis and M. L. Johnson, Saltation and stasis: a model of human growth, *Science* 258, 801–803, 1992.
9. A. Sabine, Coherence–Rupture–Regeneration as Bounded Left Kan Extension, January 2026.

Appendix: complete code

Each file is reproduced verbatim from the frozen study directory.

`kan_model.py`

```
        """Model for KAN-CHECK-2026-10-28 (frozen): CRR as a left Kan extension, rebuilt in the
current CRR form.
        1. Lan along the inclusion of completed events into time: colimit over the causal past
(computed in FinSet).
        2. Regeneration as a convex-algebra map: recursion = whole-record weighted mean on flat
contents; fails on the circle beyond a half-circle.
        3. Thresholds: Fisher-Rao diameter of the simplex is pi (sphere embedding), Bonnet-Myers
only bounds it by 2pi; half-turn of an intrinsic circle carrier.
        4. Max-entropy age weights: geometric.
        5. First-passage CV of drift-diffusion = sqrt(sigma^2/(mu*Omega)): free, not fixed by
geometry."""
        import numpy as np, itertools

        # ---------- 1. colimits in FinSet ----------
        def colimit(objs, arrows):
            """objs: {node: size}; arrows: list of (src, dst, map as list). Returns (n_classes,
class_of[(node, i)])."""
            parent = {}
            def find(x):
                while parent[x] != x: parent[x] = parent[parent[x]]; x = parent[x]
                return x
            for v, n in objs.items():
                for i in range(n): parent[(v, i)] = (v, i)
            for s, d, f in arrows:
                for i, j in enumerate(f):
                    a, b = find((s, i)), find((d, j))
                    if a != b: parent[a] = b
            cls = {k: find(k) for k in parent}; return len(set(cls.values())), cls

        def random_causal_diagram(rng, n_agents=2, n_events=6, p_msg=0.3, max_size=4):
            """Events per agent in a chain; messages add cross-agent order. F(e) finite sets; maps
along covering relations random functions."""
            ev = [(a, k) for a in range(n_agents) for k in range(n_events)]
            t = {e: e[1] + rng.uniform(0, 0.9) for e in ev}                    # event times
(strictly increasing per agent)
            cover = [((a, k), (a, k + 1)) for a in range(n_agents) for k in range(n_events - 1)]
            for e, f in itertools.permutations(ev, 2):
                if e[0] != f[0] and t[e] < t[f] and rng.random() < p_msg / n_events:
cover.append((e, f))
            size = {e: int(rng.integers(1, max_size + 1)) for e in ev}
            maps = [(e, f, list(rng.integers(0, size[f], size[e]))) for e, f in cover]
            return ev, t, cover, size, maps

        def past(ev, cover, target_events):
            """Down-closure under the covering relation."""
            pred = {e: [a for a, b in cover if b == e] for e in ev}; seen = set(); stack =
list(target_events)
            while stack:
                x = stack.pop()
                if x in seen: continue
                seen.add(x); stack += pred[x]
            return seen

        def lan_at(ev, t, cover, size, maps, agent, time):
            """Lan along inclusion events -> (agent's time line): colimit over events in the causal
past of `agent` at `time`."""
            own = [e for e in ev if e[0] == agent and t[e] <= time]
            P = past(ev, cover, own)
            objs = {e: size[e] for e in P}; arr = [m for m in maps if m[0] in P and m[1] in P]
```

```
            return colimit(objs, arr) if objs else (0, {}), P

        # ---------- 2. convex-algebra regeneration ----------
        def flat_recursion(m0, Phi, q):
            m = m0.copy()
            for p in Phi: m = q * m + (1 - q) * p
            return m

        def flat_whole(m0, Phi, q):
            n = len(Phi); w = [(1 - q) * q ** (n - 1 - j) for j in range(n)]; w0 = q ** n
            return (w0 * m0 + sum(wi * p for wi, p in zip(w, Phi))) / (w0 + sum(w))

        def ang(x): return (x + np.pi) % (2 * np.pi) - np.pi

        def circle_recursion(m0, Phi, q):
            m = m0
            for p in Phi: m = ang(m + (1 - q) * ang(p - m))           # geodesic interpolation
toward the new content
            return m

        def circle_whole(m0, Phi, q, grid=20001):
            n = len(Phi); pts = np.array([m0] + list(Phi)); w = np.array([q ** n] + [(1 - q) * q **
(n - 1 - j) for j in range(n)])
            ys = np.linspace(-np.pi, np.pi, grid); cost = (w[None, :] * ang(ys[:, None] - pts[None,
:]) ** 2).sum(1)
            y = ys[np.argmin(cost)]
            for h in (1e-3, 1e-5, 1e-7, 1e-9):                         # local refinement
                cand = y + np.linspace(-h * 300, h * 300, 601)
                c = (w[None, :] * ang(cand[:, None] - pts[None, :]) ** 2).sum(1); y =
cand[np.argmin(c)]
            return ang(y)

        def arc_span(points):
            """Length of the smallest arc containing all points."""
            x = np.sort(np.mod(points, 2 * np.pi)); gaps = np.diff(np.concatenate([x, [x[0] + 2 *
np.pi]])); return 2 * np.pi - gaps.max()

        # ---------- 3. thresholds ----------
        def fr(p, q): return 2 * np.arccos(np.clip(np.sqrt(p * q).sum(), -1, 1))

        def circle_family_length(c=0.6, n=200000):
            th = np.linspace(0, 2 * np.pi, n + 1); j = np.arange(3)
            P = (1 + c * np.cos(th[:, None] - 2 * np.pi * j[None, :] / 3)) / 3
            dP = np.gradient(P, th, axis=0); g = (dP ** 2 / P).sum(1)
            return float(np.trapezoid(np.sqrt(g), th))

        # ---------- 4. max-entropy ages ----------
        def geometric_weights(q, n):
            w = q ** np.arange(n)[::-1]; return w / w.sum()

        # ---------- 5. first passage ----------
        def fpt_cv(mu, sigma, Omega, n, seed, dt=1e-3):
            rng = np.random.default_rng(seed); c = np.zeros(n); T = np.full(n, np.nan); done =
np.zeros(n, bool); k = 0
            while not done.all():
                live = ~done; c[live] += mu * dt + sigma * np.sqrt(dt) *
rng.standard_normal(live.sum()); k += 1
                hit = live & (c >= Omega); T[hit] = k * dt; done |= hit
            return float(T.std() / T.mean())
```

run_checks.py

```
        """Holdout runner for KAN-CHECK-2026-10-28. Usage: python3 -I -B run_checks.py OUT.json
        Holdout draws: default_rng(3000) for every random instance; first-passage seeds 3001-
3006."""
        import json, sys, pathlib
        sys.path.insert(0, str(pathlib.Path(__file__).resolve().parent))
```

```
        import numpy as np
        from kan_model import *

        def rnd(x):
            if isinstance(x, (float, np.floating)): return round(float(x), 9)
            if isinstance(x, list): return [rnd(v) for v in x]
            if isinstance(x, dict): return {k: rnd(v) for k, v in x.items()}
            return x

        rng = np.random.default_rng(3000); out = {}; N = 500
        ok = 0
        for _ in range(N):
            ev, t, cover, size, maps = random_causal_diagram(rng, n_agents=1, n_events=6)
            time = rng.uniform(0, 6); (n, cls), P = lan_at(ev, t, cover, size, maps, 0, time)
            own = [e for e in ev if t[e] <= time]
            if not own: ok += (n == 0); continue
            last = max(own, key=lambda e: t[e]); ok += (n == size[last]) and len({cls[(last, i)]
for i in range(size[last])}) == size[last]
        out["K1_chain_lan_equals_last"] = dict(n=N, ok=ok)
        ok = 0; others = 0
        for _ in range(N):
            ev, t, cover, size, maps = random_causal_diagram(rng, n_agents=3, n_events=5,
p_msg=0.6)
            time = rng.uniform(0, 5); (n1, _), P = lan_at(ev, t, cover, size, maps, 0, time);
others += any(e[0] != 0 for e in P)
            size2 = dict(size)
            for e in ev:
                if e not in P: size2[e] = int(rng.integers(1, 6))
            maps2 = [(s, d, f if (s in P and d in P) else list(rng.integers(0, size2[d],
size2[s]))) for s, d, f in maps]
            (n2, _), _ = lan_at(ev, t, cover, size2, maps2, 0, time); ok += (n1 == n2)
        out["K2_gating"] = dict(n=N, ok=ok, pasts_with_other_agents=others)
        errs = []
        for _ in range(1000):
            q = rng.uniform(0.05, 0.95); m0 = rng.normal(size=4); Phi = [rng.normal(size=4) for _
in range(int(rng.integers(1, 15)))]
            errs.append(float(np.abs(flat_recursion(m0, Phi, q) - flat_whole(m0, Phi, q)).max()))
        out["K3_flat_algebra"] = dict(n=1000, max_err=max(errs))
        inside, outside = [], []
        for _ in range(800):
            q = rng.uniform(0.3, 0.9); n = int(rng.integers(2, 6))
            if rng.random() < 0.5:
                c = rng.uniform(-np.pi, np.pi); width = rng.uniform(0.2, 0.9 * np.pi); pts = ang(c
+ rng.uniform(-width / 2, width / 2, n + 1))
            else:
                pts = rng.uniform(-np.pi, np.pi, n + 1)
            d = float(abs(ang(circle_recursion(pts[0], pts[1:], q) - circle_whole(pts[0], pts[1:],
q))))
            (inside if arc_span(pts) < np.pi else outside).append(d)
        out["K4_circle"] = dict(n_inside=len(inside), max_inside=max(inside),
n_outside=len(outside),
                                frac_outside_differ=float(np.mean(np.array(outside) > 1e-3)),
max_outside=max(outside))
        worst = 0.0
        for _ in range(20000):
            p, qq = rng.dirichlet(np.ones(6) * 0.3), rng.dirichlet(np.ones(6) * 0.3); worst =
max(worst, fr(p, qq))
        out["K5_simplex_diameter"] = dict(max_sampled=worst, vertex_pair=fr(np.eye(6)[0], np.eye(6)
[1]), bonnet_myers_bound=2 * np.pi)
        L = circle_family_length(); out["K6_carrier_threshold"] = dict(circumference=L, half_turn=L
/ 2, Omega=2 / L)
        n, q = 8, 0.7; g = geometric_weights(q, n); ages = np.arange(n)[::-1]; Hg = float(-(g *
np.log(g)).sum()); worse = 0
        for _ in range(2000):
            v = rng.normal(size=n); A = np.vstack([np.ones(n), ages]); v -= A.T @
np.linalg.lstsq(A.T, v, rcond=None)[0]
            eps = 0.5 * g.min() / np.abs(v).max(); p = g + eps * v * rng.uniform(0.1, 1); worse +=
bool(-(p * np.log(p)).sum() < Hg)
        out["K7_maxent_ages"] = dict(n=2000, geometric_wins=worse)
```

```
        rows = []
        for k, seed in enumerate(range(3001, 3007)):
            mu, s, Om = float(rng.uniform(0.5, 2)), float(rng.uniform(0.2, 1.2)),
float(rng.uniform(1, 7))
            sim = fpt_cv(mu, s, Om, 4000, seed); rows.append(dict(mu=mu, sigma=s, Omega=Om,
cv_sim=sim, cv_theory=float(np.sqrt(s * s / (mu * Om)))))
        out["K8_cv_free"] = rows
        pathlib.Path(sys.argv[1]).write_text(json.dumps(rnd(out), indent=1, sort_keys=True) + "\n")
```

score.py

```
        """Frozen scorer for KAN-CHECK-2026-10-28. Usage: python3 -I -B score.py outcomes.json >
verdict.json"""
        import json, math, sys
        o = json.load(open(sys.argv[1])); c = o["K4_circle"]; k8 = o["K8_cv_free"]; th =
[r["cv_theory"] for r in k8]
        checks = {
         "V0_harness_counts": o["K1_chain_lan_equals_last"]["n"] == 500 and o["K2_gating"]["n"] ==
500 and c["n_inside"] + c["n_outside"] == 800,
         "K1_chain_lan_equals_last": o["K1_chain_lan_equals_last"]["ok"] == 500,
         "K2_gating": o["K2_gating"]["ok"] == 500,
         "K3_flat_algebra": o["K3_flat_algebra"]["max_err"] < 1e-12,
         "K4_circle_within_half": c["max_inside"] < 1e-6,
         "K4_circle_beyond_half": c["frac_outside_differ"] >= 0.15 and c["max_outside"] > 1.0,
         "K5_diameter_pi": o["K5_simplex_diameter"]["max_sampled"] <= math.pi + 1e-12 and
abs(o["K5_simplex_diameter"]["vertex_pair"] - math.pi) < 1e-9 and o["K5_simplex_diameter"]
["bonnet_myers_bound"] > math.pi + 1,
         "K6_carrier_threshold": abs(o["K6_carrier_threshold"]["circumference"] - 2.794018) < 1e-4,
         "K7_maxent_ages": o["K7_maxent_ages"]["geometric_wins"] == 2000,
         "K8_cv_free": all(abs(r["cv_sim"] / r["cv_theory"] - 1) <= 0.05 for r in k8) and max(th) /
min(th) > 2,
        }
        verdict = "VOID" if not checks["V0_harness_counts"] else ("RESULT" if all(checks.values())
else "FAIL")
        json.dump({"study_id": "KAN-CHECK-2026-10-28", "checks": checks, "verdict": verdict},
sys.stdout, indent=1, sort_keys=True); print()
```

controls.py

```
        """Controls for KAN-CHECK-2026-10-28, scored before the freeze. Usage: python3 -I -B
controls.py > controls.json
        pos_pushout: the colimit routine returns the textbook pushout of {0,1} <- {*} -> {0,1,2} (4
elements).
        neg_mismatched_weights: a recursion with weight 1/2 compared against geometric whole-record
weights with q = 0.8 must FAIL the flat law.
        neg_wrapless_circle: averaging raw angles without wrapping must FAIL the within-half-circle
agreement for arcs crossing +-pi."""
        import json, sys, pathlib
        sys.path.insert(0, str(pathlib.Path(__file__).resolve().parent))
        import numpy as np
        from kan_model import colimit, flat_whole, circle_whole, ang
        n, _ = colimit({"A": 1, "B": 2, "C": 3}, [("A", "B", [0]), ("A", "C", [0])])
        rng = np.random.default_rng(7000); errs = []
        for _ in range(200):
            m0 = rng.normal(size=3); Phi = [rng.normal(size=3) for _ in range(5)]; m = m0.copy()
            for p in Phi: m = 0.5 * m + 0.5 * p
            errs.append(np.abs(m - flat_whole(m0, Phi, 0.8)).max())
        pts = ang(np.pi + np.array([-0.3, 0.2, 0.1, -0.1]))       # a tight arc straddling +-pi
        raw = 0.8 ** 3 * pts[0] + sum((1 - 0.8) * 0.8 ** (2 - j) * p for j, p in
enumerate(pts[1:]))
        raw /= 0.8 ** 3 + sum((1 - 0.8) * 0.8 ** (2 - j) for j in range(3))
        out = {"pos_pushout": {"size": n, "pass": n == 4},
               "neg_mismatched_weights": {"max_err": float(max(errs)), "fails_law": bool(max(errs)
> 1e-3)},
               "neg_wrapless_circle": {"diff": float(abs(ang(raw - circle_whole(pts[0], pts[1:],
```

```
0.8)))), "fails": bool(abs(ang(raw - circle_whole(pts[0], pts[1:], 0.8))) > 1e-3)}}
        out["controls_behave"] = out["pos_pushout"]["pass"] and out["neg_mismatched_weights"]
["fails_law"] and out["neg_wrapless_circle"]["fails"]
        print(json.dumps(out, indent=1, sort_keys=True))
```

run.sh

```
        #!/bin/sh
        # usage: sh run.sh RUNS_DIR   (after the freeze and anchor; holdout draws
default_rng(3000), seeds 3001-3006)
        set -e
        H="$(cd "$(dirname "$0")" && pwd)"; R="$1"; mkdir -p "$R"
        python3 -I -B "$H/run_checks.py" "$R/outcomes.json"
        python3 -I -B "$H/score.py" "$R/outcomes.json" > "$R/verdict.json"
```

figure.py

```
        """Figure 1 (illustrative; fresh draws default_rng(4000), not the holdout): disagreement
between recursive and
        whole-record regeneration on the circle against the arc spanned by the contents. Usage:
python3 -B figure.py"""
        import numpy as np
        import matplotlib; matplotlib.use("Agg")
        import matplotlib.pyplot as plt
        from kan_model import circle_recursion, circle_whole, arc_span, ang
        rng = np.random.default_rng(4000); span, diff = [], []
        for _ in range(1500):
            q = rng.uniform(0.3, 0.9); n = int(rng.integers(2, 6)); pts = rng.uniform(-np.pi,
np.pi, n + 1)
            if rng.random() < 0.5: pts = ang(rng.uniform(-np.pi, np.pi) + rng.uniform(-1, 1, n + 1)
* rng.uniform(0.1, 1.6))
            span.append(arc_span(pts)); diff.append(abs(ang(circle_recursion(pts[0], pts[1:], q) -
circle_whole(pts[0], pts[1:], q))))
        span, diff = np.array(span), np.maximum(np.array(diff), 1e-10)
        fig, ax = plt.subplots(figsize=(7.2, 4.0), dpi=200)
        ax.semilogy(span, diff, ".", ms=3, color="#4A7A2C", alpha=0.6)
        ax.axvline(np.pi, color="#A3402B", lw=1.2, ls="--"); ax.text(np.pi + 0.05, 1e-8, "half-
circle (π)", color="#A3402B", fontsize=9)
        ax.set_xlabel("arc spanned by the record (rad)"); ax.set_ylabel("|recursive seed − whole-
record Fréchet mean| (rad)")
        for sp in ("top", "right"): ax.spines[sp].set_visible(False)
        ax.grid(color="#e6e2d8", lw=0.6); fig.tight_layout(); fig.savefig("../fig_circle.png",
facecolor="white")
```